# Coloriage de graphes – Étape 1 : cadrage

Nov 9, 2026 · @claude

## Décisions de l'étape 1

L'étape 1 fixe les deux exemples du rapport et le code C commun sur lequel B2 (glouton) et B3 (backtracking) construisent leurs algorithmes.

| Sujet | Décision |
|---|---|
| Exemple simple (§3) | Graphe « prisme » à 6 sommets (A à F) et 9 arêtes, $\chi(G) = 3$ |
| Exemple réel (§4) | Emploi du temps de 7 examens : 3 créneaux suffisent et sont nécessaires |
| Représentation du graphe | Matrice d'adjacence n × n en `unsigned char`, sommets numérotés de 0 à n−1 |
| Couleurs | Entiers de 0 à K−1, la valeur −1 signifie « non colorié » |
| Fichiers de graphes | Format DIMACS (`.col`), le standard des benchmarks de coloriage |
| Graphes de test | Générés à partir d'une graine : identiques sur toutes les machines (Windows, Linux, Mac) |
| Mesures | Un seul format CSV pour tout le groupe, une ligne par exécution |
| Compilation | `gcc -O2 -fopenmp`, avec les mêmes options pour les versions séquentielle et parallèle |

## Exemple simple : le graphe prisme (§3)

Le graphe prisme se colorie avec 3 couleurs, et 3 est le minimum : $\chi(G) = 3$.

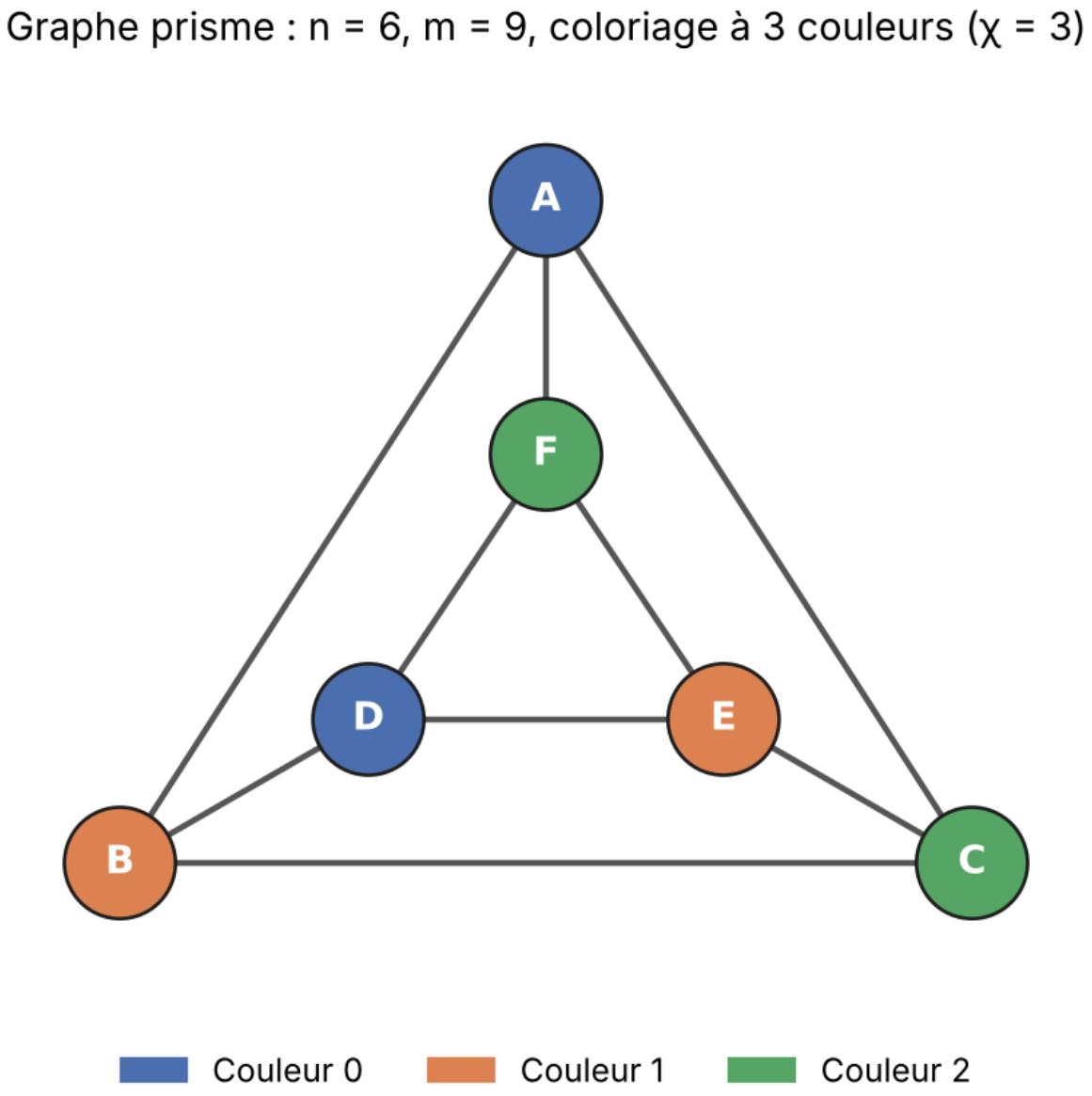
**Sommets** : $V = \{A, B, C, D, E, F\}$, donc $n = 6$.

**Arêtes** : deux triangles A-B-C et D-E-F, reliés par A-F, B-D et C-E. $E = \{AB, AC, BC, DE, DF, EF, AF, BD, CE\}$, donc $m = 9$. Chaque sommet a 3 voisins : le graphe est 3-régulier et son degré maximal vaut $\Delta = 3$.

**Matrice d'adjacence** (1 = arête entre les deux sommets) :

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 1 |
| B | 1 | 0 | 1 | 1 | 0 | 0 |
| C | 1 | 1 | 0 | 0 | 1 | 0 |
| D | 0 | 1 | 0 | 0 | 1 | 1 |
| E | 0 | 0 | 1 | 1 | 0 | 1 |
| F | 1 | 0 | 0 | 1 | 1 | 0 |

**Coloriage proposé** : couleur 0 pour A et D, couleur 1 pour B et E, couleur 2 pour C et F.



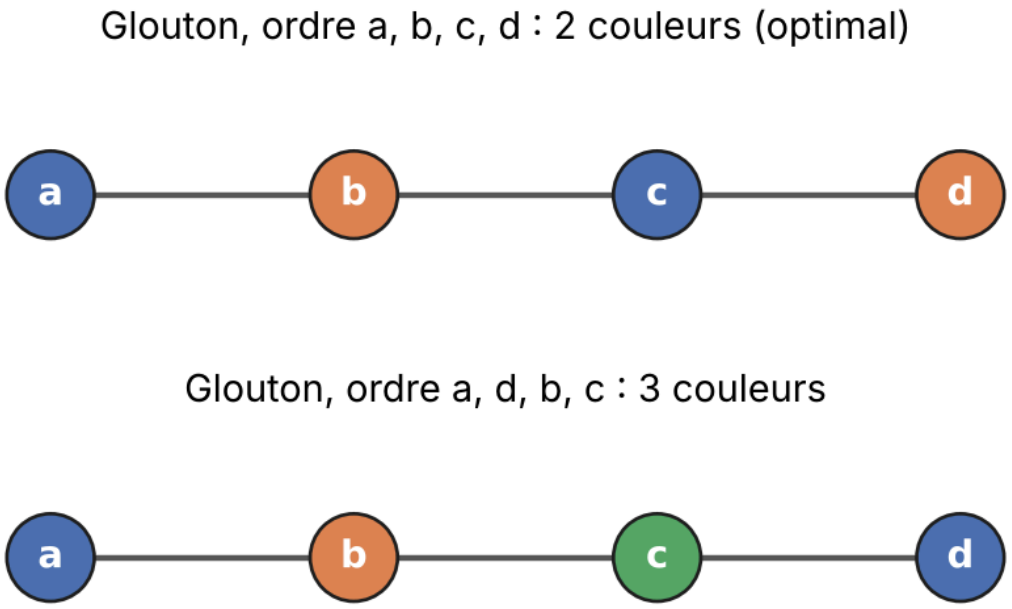

**Vérification** : les 9 arêtes relient toujours deux couleurs différentes.

| Arête | Couleurs des extrémités | Valide |
|---|---|---|
| A-B | 0 et 1 | oui |
| A-C | 0 et 2 | oui |
| B-C | 1 et 2 | oui |
| D-E | 0 et 1 | oui |
| D-F | 0 et 2 | oui |
| E-F | 1 et 2 | oui |
| A-F | 0 et 2 | oui |
| B-D | 1 et 0 | oui |
| C-E | 2 et 1 | oui |

**Nombre de couleurs utilisées** : 3.

**Preuve d'optimalité** : A, B et C sont deux à deux adjacents (un triangle). Il faut donc déjà 3 couleurs différentes pour eux, d'où $\chi(G) \geqslant 3$. Le coloriage ci-dessus en utilise exactement 3, donc $\chi(G) = 3$.

**Pour la trace d'exécution (§6)** : le glouton dans l'ordre A, B, C, D, E, F retrouve ce coloriage à 3 couleurs. Le backtracking avec K = 2 échoue dès le sommet C, après des retours en arrière visibles ; avec K = 3, il réussit sans retour. Le fichier `chemin_p4.col` (chemin a-b-c-d) sert à montrer que le glouton dépend de l'ordre : 2 couleurs dans l'ordre a, b, c, d, mais 3 dans l'ordre a, d, b, c.




## Exemple réel : l'emploi du temps des examens (§4)

Les 7 examens tiennent en 3 créneaux sans qu'aucun étudiant n'ait deux épreuves en même temps, et 2 créneaux ne suffiraient pas.

**Contexte** : l'école doit placer 7 examens dans le moins de créneaux possible. Deux examens qui ont un étudiant en commun ne peuvent pas avoir lieu en même temps.

**Modélisation**

| Élément du graphe | Sens dans le problème réel |
|---|---|
| Sommet | Un examen |
| Arête | Au moins un étudiant est inscrit aux deux examens |
| Couleur | Un créneau horaire |
| Coloriage propre | Aucun étudiant n'a deux examens dans le même créneau |
| $\chi(G)$ | Nombre minimal de créneaux |

**Inscriptions** (données inventées pour l'exemple)

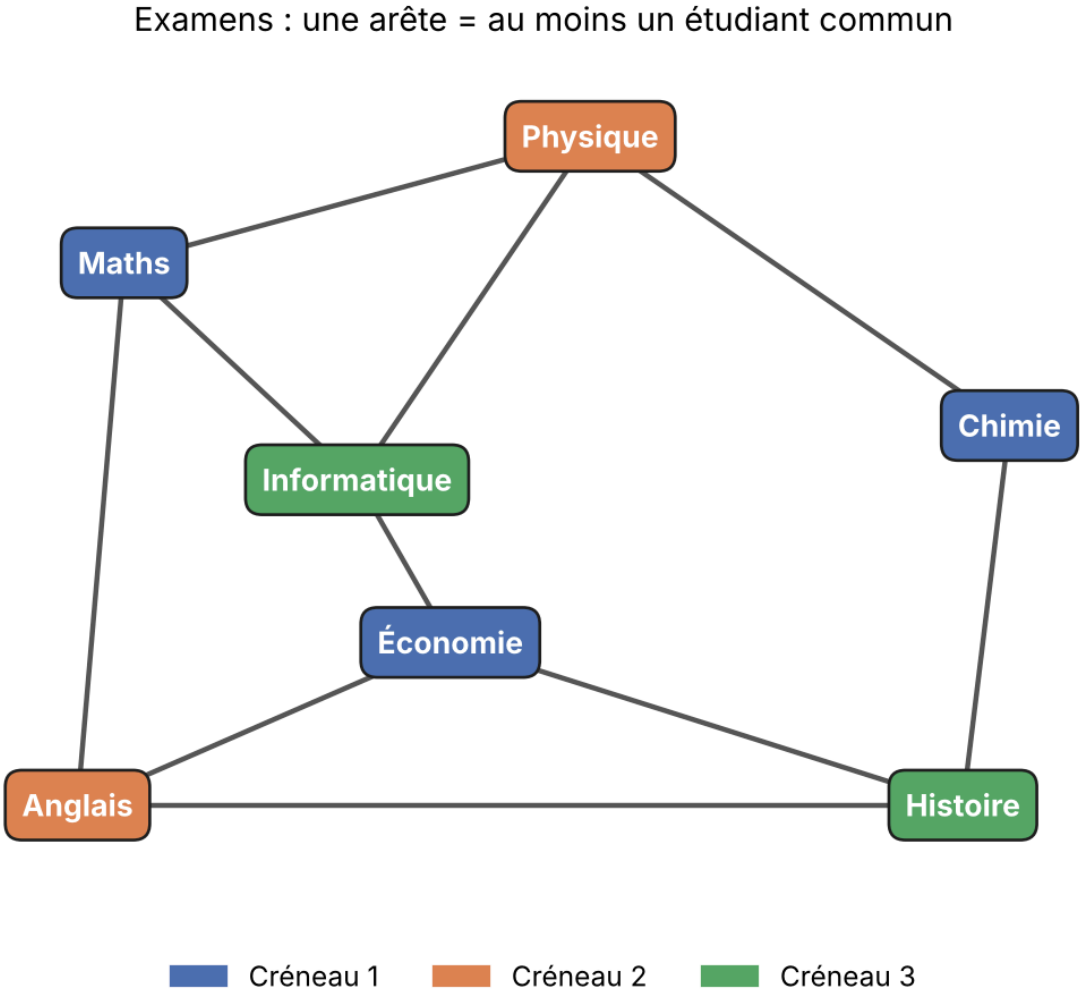
| Étudiant | Examens passés |
|---|---|
| Étudiant 1 | Maths, Physique, Informatique |

| Étudiant | Examens passés |
|---|---|
| Étudiant 2 | Maths, Anglais |
| Étudiant 3 | Physique, Chimie |
| Étudiant 4 | Informatique, Économie |
| Étudiant 5 | Anglais, Économie, Histoire |
| Étudiant 6 | Chimie, Histoire |

**Graphe obtenu** : n = 7 sommets et m = 10 arêtes. Les arêtes sont Maths-Physique, Maths-Informatique, Physique-Informatique, Maths-Anglais, Physique-Chimie, Informatique-Économie, Anglais-Économie, Anglais-Histoire, Économie-Histoire et Chimie-Histoire.

**Solution** (trouvée par le glouton dans l'ordre du fichier, et vérifiée par le programme de test)

| Créneau | Examens |
|---|---|
| Créneau 1 (couleur 0) | Maths, Économie, Chimie |
| Créneau 2 (couleur 1) | Physique, Anglais |
| Créneau 3 (couleur 2) | Informatique, Histoire |




**Interprétation** : l'étudiant 1 passe Maths, Physique et Informatique. Ces trois examens forment un triangle et exigent 3 créneaux distincts. 3 créneaux sont donc le minimum, et la solution ci-dessus l'atteint.

## Code commun

Le code commun compile sans avertissement et passe ses 27 tests (`make test`). Chaque membre du groupe lance ces tests sur sa machine avant de commencer l'étape 2.

### Contenu du dossier

| Fichier | Rôle |
|---|---|
| `graphe.h` / `graphe.c` | Matrice d'adjacence, lecture et écriture DIMACS, générateur reproductible, vérification des colorations |
| `mesures.h` / `mesures.c` | Chronomètre, CSV des mesures, trace des tâches pour les diagrammes de Gantt |

| Fichier | Rôle |
|---|---|
| `generer.c` | Outil : `./generer <n> <densité> <graine> <fichier.col>` |
| `test_commun.c` | 27 tests sur les exemples, le générateur, les CSV et OpenMP |
| `graphes/*.col` | `exemple_simple.col`, `examens.col`, `chemin_p4.col` |
| `Makefile`, `README.md` | Compilation et mode d'emploi |

Le dossier `figures/` contient les trois figures de cette fiche et le script Python `dessiner_exemples.py` qui les redessine à partir des fichiers `.col` (pour le rapport de B1).

### Conventions

- Sommets numérotés de 0 à n−1, couleurs de 0 à K−1, `NON_COLORE` (−1) pour un sommet pas encore colorié.
- Matrice en `unsigned char` (1 octet par case) : un graphe de 10 000 sommets occupe 100 Mo.
- Un résultat est toujours vérifié par `coloration_est_valide`, en dehors du chronométrage.
- Personne ne modifie `graphe.c` ou `mesures.c` seul : un changement se décide en groupe.

### Fonctions disponibles

| Fonction | Rôle | Coût |
|---|---|---|
| `adjacent(g, u, v)` | 1 si u et v sont reliés | O(1) |
| `graphe_charger(chemin)` / `graphe_sauver(g, chemin)` | Lire ou écrire un fichier DIMACS | O(n²) |
| `graphe_aleatoire(n, densité, graine)` | Graphe aléatoire, identique sur toutes les machines | O(n²) |
| `graphe_degre(g, v)` / `graphe_degre_max(g)` | Degré d'un sommet, degré maximal Δ | O(n) / O(n²) |
| `graphe_afficher(g)` | Résumé, et matrice si n ⩽ 20 | O(n²) |
| `coloration_creer(n)` / `coloration_copier(c, n)` | Tableau de couleurs, copie indépendante pour une tâche | O(n) |

| Fonction | Rôle | Coût |
|---|---|---|
| `coloration_est_valide(g, c)` | 1 si tous les sommets sont coloriés sans conflit | O(n²) |
| `coloration_nb_conflits(g, c)` / `coloration_nb_couleurs(c, n)` | Diagnostic d'une coloration | O(n²) / O(n) |
| `coloration_afficher(g, c)` | Sommets regroupés par couleur | O(n²) |
| `chrono()` | Temps en secondes ( `omp_get_wtime` ) | O(1) |
| `mesure_ecrire(...)` | Ajoute une ligne au CSV des mesures | O(1) |
| `trace_demarrer` , `trace_ajouter` , `trace_ecrire_csv` | Trace des tâches pour le Gantt, utilisable depuis plusieurs threads | O(1) par tâche |

**Compilation**

- Windows avec Code::Blocks : ouvrir `coloriage.cbp` , choisir la cible `test_commun` , puis F9. Les options `-O2 -std=c11 -fopenmp` sont déjà réglées dans le projet.
- Linux ou WSL : `make` puis `make test` .
- Mac : le `gcc` d'Apple ne gère pas OpenMP. Installer `brew install gcc` , puis `make CC=gcc-14` .

Le test « même nombre d'arêtes que sur les autres machines » vérifie que `graphe_aleatoire(200, 0.3, 42)` donne 5 948 arêtes. Un autre résultat signale un problème de compilateur à régler avant de mesurer.

## Formats de fichiers et graphes de test

Tout le groupe utilise les mêmes formats, pour que les résultats de B2 et B3 se fusionnent sans retouche dans les courbes et le rapport.

**Graphes : format DIMACS** ( `.col` , sommets numérotés à partir de 1 dans le fichier). C'est le format des benchmarks publics de coloriage : B3 pourra tester son backtracking sur des graphes dont χ est connu.

```
c commentaire libre
p edge 6 9
c nom 1 A
e 1 2
e 1 3
```

**Mesures : un CSV, une ligne par exécution**, écrit par `mesure_ecrire`. La version séquentielle s'écrit `seq` avec 1 thread, la version parallèle `par` avec 1, 2, 4 ou 8 threads.

```
algo,version,graphe,n,m,threads,essai,temps_s,nb_couleurs
glouton,seq,G1,2000,998698,1,1,0.018000,224
glouton,par,G1,2000,998698,4,1,0.006000,224
```

**Traces pour les diagrammes de Gantt : un CSV, une ligne par tâche**, écrit par `trace_ecrire_csv`. Les temps partent de l'appel à `trace_demarrer`.

```
thread,debut_ms,fin_ms,duree_ms,etiquette
0,0.0120,3.4100,3.3980,v=2 c=0
1,0.0150,5.0200,5.0050,v=2 c=1
```

**Graphes de mesure pour le glouton (B2)** : densité 0,5, créés en mémoire avec `graphe_aleatoire`. Ils ne sont pas enregistrés en fichier, car G3 ferait à lui seul environ 300 Mo de texte. Le nombre d'arêtes sert de contrôle : chacun doit retrouver exactement ces valeurs.

| Graphe | n | Graine | m (contrôle) | Mémoire | Glouton séquentiel simple |
|---|---|---|---|---|---|
| G1 | 2 000 | 1 | 998 698 | 4 Mo | 0,02 s |
| G2 | 5 000 | 2 | 6 249 474 | 25 Mo | 0,11 s |
| G3 | 10 000 | 3 | 25 002 185 | 100 Mo | 0,46 s |
| G4 | 15 000 | 4 | 56 247 329 | 225 Mo | 0,99 s |

Les temps ont été mesurés avec un glouton simple sur la machine de test : ce sont des ordres de grandeur. B2 confirme ou ajuste ces tailles à l'étape 2.

**Graphes de mesure pour le backtracking (B3)** : 4 petits graphes de densité 0,5, enregistrés dans `graphes/BT1.col` à `graphes/BT4.col`. Ils sont petits parce que le temps croît de façon exponentielle : à n = 34, les trois graines essayées dépassent déjà 60 s.

| Graphe | n | m | Graine | χ | Ts sur le PC de mesure (moyenne de 5) | Ce qui coûte le plus |
|---|---|---|---|---|---|---|
| BT1 | 24 | 150 | 1 | 7 | 1,035 s | Prouver que 6 couleurs sont impossibles (1,02 s) |
| BT2 | 26 | 176 | 1 | 7 | 1,955 s | Prouver que 6 couleurs sont impossibles (1,94 s) |
| BT3 | 30 | 221 | 3 | 7 | 5,399 s | Preuve à 6 couleurs (4,62 s), puis recherche à 7 couleurs (0,74 s) |

| Graphe | n | m | Graine | χ | Ts sur le PC de mesure (moyenne de 5) | Ce qui coûte le plus |
|---|---|---|---|---|---|---|
| BT4 | 32 | 253 | 6 | 7 | 9,229 s | Trouver la solution à 7 couleurs (8,88 s) |

Ces temps Ts sont la référence pour l'accélération. Ils ont été mesurés le 9 novembre 2026 sur le PC du binôme B3 (12 threads), qui servira aussi aux mesures parallèles. Les 5 répétitions de chaque graphe varient de moins de 4 %.

Une première série BT1 à 2,70 s, faussée par le mode d'énergie de Windows, a été écartée : dans `mesures.csv`, seule la dernière série BT1 compte. Les nombres de nœuds sont identiques sur les deux machines testées, ce qui confirme que tout le groupe calcule sur les mêmes graphes. Le fichier `graphes/myciel4.col` (χ = 5 connu) sert à vérifier que le programme trouve le bon résultat.

## Répartition et actions pour l'étape 2

L'étape 1 est terminée quand chacun a compilé le code commun et que le groupe a validé les deux exemples. Chaque binôme démarre ensuite sa partie.

| Binôme | Parties du sujet | Point de départ de l'étape 2 |
|---|---|---|
| B1 – Théorie et rapport | §1 à §7, §11, assemblage du rapport et du PowerPoint | Rédiger §1 à §4 avec les deux exemples ci-dessus, puis les pseudo-codes et les traces (§6) |
| B2 – Glouton | §8.1, §9.1, mesures et courbes du glouton | `glouton_seq.c` testé sur les trois fichiers `.col`, puis chronométré sur G1 à G4 |
| B3 – Backtracking | §8.2, §9.2, mesures et Gantt du backtracking | `backtrack_seq.c` avec calcul de χ, vérifié sur l'exemple simple (χ = 3), puis calibrage de BT1 à BT4 |

### À faire pour clore l'étape 1

- [ ] Chaque membre lance `make test` et obtient « 27 tests, 0 échec(s) »
- [ ] Le code est déposé dans un espace partagé (dépôt Git ou dossier commun)
- [ ] Le groupe valide les deux exemples : graphe prisme et emploi du temps des examens
- [ ] B2 confirme ou ajuste les tailles G1 à G4
- [x] ~~B3 calibre BT1 à BT4 et complète leur tableau~~
- [ ] B2 et B3 relisent les pseudo-codes de B1 dès qu'ils sont écrits